Name: Alan Souza
2DAM - SGE
28/10/26

# Actividades Evaluables parte 1

IES DR FAUSTI BETERA

Name: Alan Souza
2DAM - SGE 28/10/26

# Índice

Act 1 – Clonar y Copiar Una Lista....3
Act 1.1 – ¿Shallow copy y deep copy?....4
Act 1.2 Añadir y Quitar un elemento....5
#● Afegir un element a una llista....5
Act 1.4 Convertir palabras de una cadena en una lista....7
Act 1.5 Comentarios....8
....8
Act 2.0 recibir dos números y devolver la suma....9
Act 2.1 recibir una lista y modificarla....10
Act 2.2 crear una copia y no modificar la original....11
Act 3.0 usuarios y contraseñas. #ACTIVITAT 03....12
Act 4.0 is, not and in en python....13
Act 5.0 Consola y Sobrecarga....14
Act 6.0 Ordenar y Key....15
Act 7.0 Programación orientada a objetos....16

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 1 – Clonar y Copiar Una Lista

```python
# Name: Alan Souza -- 2DAM
# UD00. Actividad Evaluable 01


#4. ACTIVITAT 01
#Prepara amb un exemple on expliques com fer en Python 3:

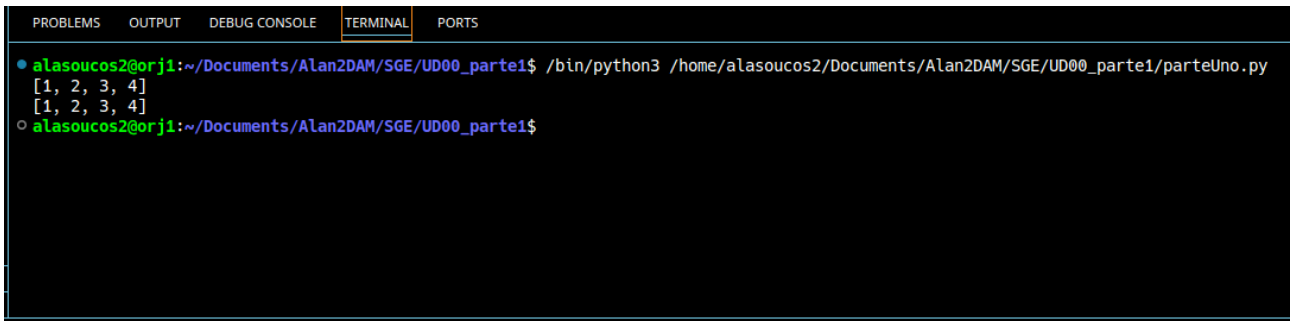
#⬤ Clonar una llista.

Lista1 = [1, 2, 3, 4]

#copy() crea una copia de la lista.
lista2 = lista1.copy()

print(lista1)
print(lista2)
```

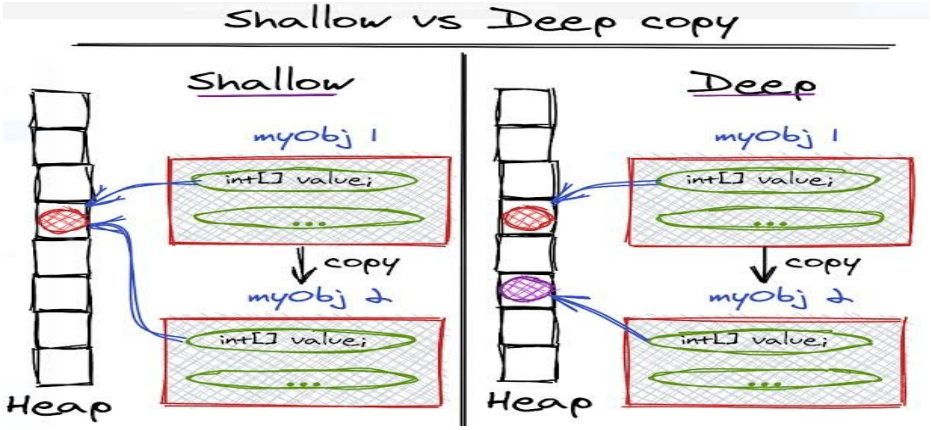
Terminal:



Name: Alan Souza
2DAM - SGE 28/10/26

# Act 1.1 – ¿Shallow copy y deep copy?

OBS: La diferencia principal entre una copia superficial (*shallow copy*) y una copia profunda (*deep copy*) radica en cómo manejan los objetos anidados (como listas dentro de listas o diccionarios dentro de diccionarios).




```python
#◯ ¿Quina és la diferència en Python entre “shallow copy” i “deep copy”?

#Shallow copy = copia superficial.
lista2 = lista1.copy()

#Copia la estructura exterior, pero los objetos que están dentro pueden seguir siendo compartidos.

#Deep copy = copia profunda.

import copy
lista2 = copy.deepcopy(lista1)

#Crea una copia independiente también de los elementos internos.
```

Terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

 /bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/parteUno.py
● alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$ /bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/parteUno.py
  [1, 2, 3, 4]
  [1, 2, 3, 4]
  [1, 2, 3, 4]
  [1, 2, 3, 4]
○ alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$
```

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 1.2 Añadir y Quitar un elemento

```python
#🟢 Afegir un element a una llista.

frutas = ["manzana", "pera"]
frutas.append("plátano")

print(frutas)

#append() añade un elemento al final.

#🟢 Llevar un element a una llista.

frutas = ["manzana", "pera", "plátano"]
frutas.remove("pera")

print(frutas)

#remove() elimina el elemento indicado.
```

Terminal:



Name: Alan Souza
2DAM - SGE 28/10/26

# Act 1.3 Crear una lista con los 4 últimos elementos

```python
#🟢 Crear una nova llista amb els 4 últims elements d'una llista.

numeros = [1, 2, 3, 4, 5, 6, 7, 8]

ultimos = numeros[-4:]

print(ultimos)

#-4: significa: Desde los cuatro últimos elementos hasta el final.
```

Terminal:




Resultado [5, 6, 7, 8]

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 1.4 Convertir palabras de una cadena en una lista

```
#🟢 Convertir les paraules d'una cadena (separades per espai) a una llista.

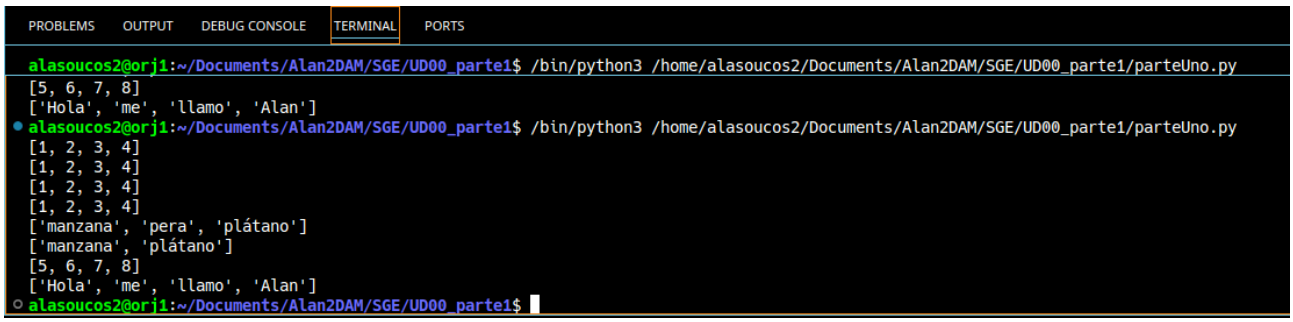
texto = "Hola me llamo Alan"

palabras = texto.split()

print(palabras)

#split() separa el texto utilizando los espacios.
```

Terminal:




Resultado: ["Hola", "me", "llamo", "Alan"]

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 1.5 Comentarios

```
#⬤ Comentaris amb una línia.

# Se utiliza #: para hacer un comentario

#⬤ Comentaris multilínia.

#Para hacer un comentario en varias líneas en Python,
# puedes usar el símbolo # al inicio de cada línea que quieras comentar
# o envolver el bloque de código entre comillas triples (''' o """).
```

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 2.0 recibir dos números y devolver la suma

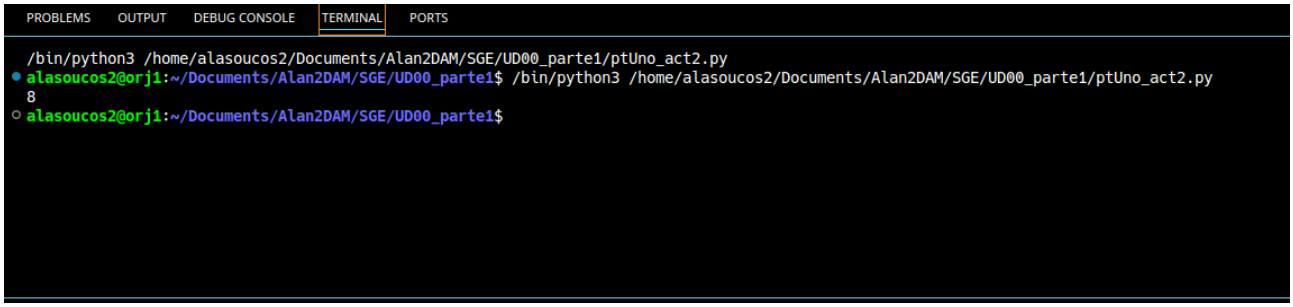
```python
#● Reva 2 números i torne la suma.

def sumar(a, b):
return a + b


resultado = sumar(5, 3)

print(resultado)
```

Terminal:




Resultado: 8

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 2.1 recibir una lista y modificarla

```
#🟢 Reva una llista i modifique eixa mateixa llista (referència) doblant els valors de tots els
#elements. No ha de retornar res.

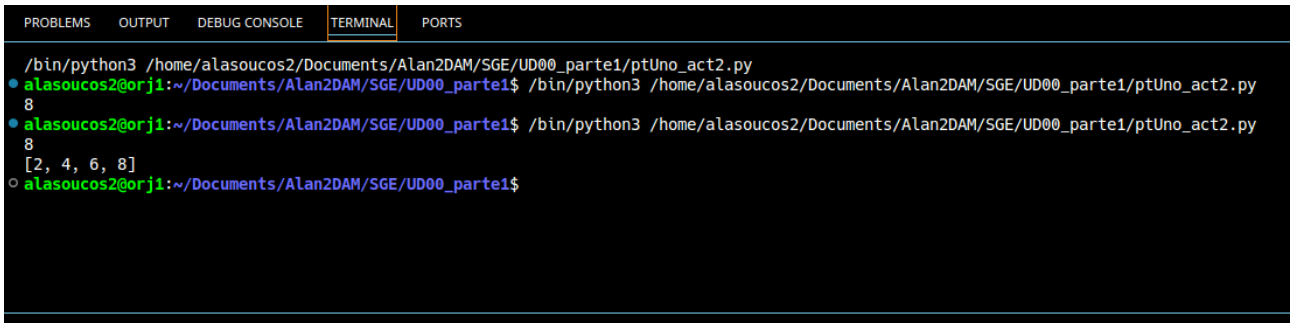
def doblar_lista(lista):
for i in range(len(lista)):
lista[i] = lista[i] * 2


numeros = [1, 2, 3, 4]

doblar_lista(numeros)

print(numeros)
```

Terminal:




Resultado: [2, 4, 6, 8]

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 2.2 crear una copia y no modificar la original

```python
#🟢 Reva una llista i torne una còpia de la llista mateixa llista (referència) doblant els valors de
#tots els elements. La llista original no hi ha de modificar-se

def doblar_lista_copia(lista):
copia = lista.copy()

for i in range(len(copia)):
copia[i] = copia[i] * 2

return copia


numeros = [1, 2, 3, 4]

resultado = doblar_lista_copia(numeros)

print("Original:", numeros)
print("Copia:", resultado)
```

Terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

/bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/ptUno_act2.py
alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$ /bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/ptUno_act2.py
8
alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$ /bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/ptUno_act2.py
8
[2, 4, 6, 8]
alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$ /bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/ptUno_act2.py
8
[2, 4, 6, 8]
Original: [1, 2, 3, 4]
Copia: [2, 4, 6, 8]
alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$
```

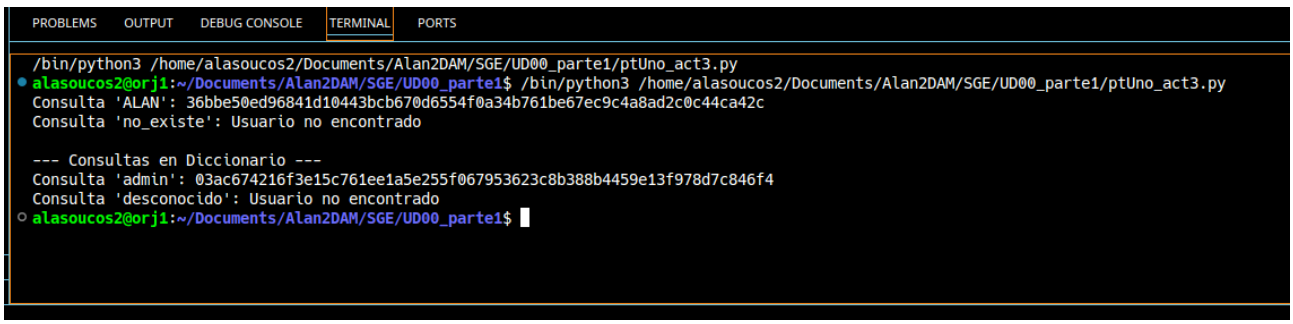
Resultado: Original: [1, 2, 3, 4] Copia: [2, 4, 6, 8]

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 3.0 usuarios y contraseñas.

```python
#ACTIVITAT 03
import hashlib
#Función auxiliar para convertir texto plano a Hash SHA-256
def generar_hash(password: str) -> str:
return hashlib.sha256(password.encode("utf-8")).hexdigest()
#Datos iniciales (5 usuarios y contraseñas)
datos_raw = [
("admin", "1234"),
("ALAN", "abcde"),
("FRAN", "pass2024"),
("AITOR", "qwerty"),
("ADRIA", "secret"),
]
#Estructura basada en listas
lista_usuarios = []
for usuario, pwd in datos_raw:
lista_usuarios.append([usuario, generar_hash(pwd)])
def consultar_lista(user_buscado):
for u, hash_p in lista_usuarios:
if u == user_buscado:
return hash_p
return "Usuario no encontrado"
#Estructura basada en DICCIONARIOS
dict_usuarios = {usuario: generar_hash(pwd) for usuario, pwd in datos_raw}
def consultar_diccionario(user_buscado):
return dict_usuarios.get(user_buscado, "Usuario no encontrado")
print("\n--- Consultas en Listas ---")
print("Consulta 'ALAN':", consultar_lista("ALAN"))
print("Consulta 'no_existe':", consultar_lista("no_existe"))
print("\n--- Consultas en Diccionario ---")
print("Consulta 'admin':", consultar_diccionario("admin"))
print("Consulta 'desconocido':", consultar_diccionario("desconocido"))
```

Terminal:



Name: Alan Souza
2DAM - SGE 28/10/26

# Act 4.0 is, not and in en python

```python
#Explica amb exemples com funcionen els operadors “is”, “not”, “in” en Python 3.

# Operador 'is' (Identidad de memoria)
a = [1, 2, 3]
b = [1, 2, 3]
c = a

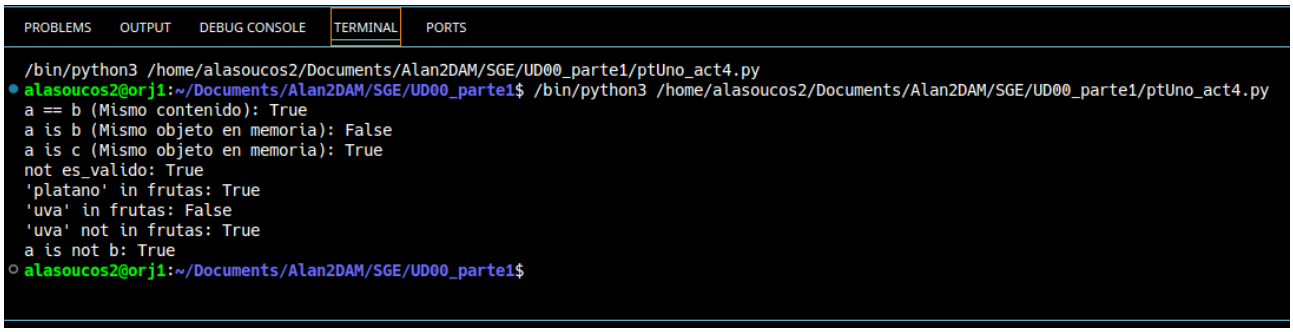
print(f"a == b (Mismo contenido): {a == b}") # True
print(f"a is b (Mismo objeto en memoria): {a is b}") # False
print(f"a is c (Mismo objeto en memoria): {a is c}") # True

# Operador 'not' (Negación lógica)
es_valido = False
print(f"not es_valido: {not es_valido}") # True

# Operador 'in' (Pertenencia)
frutas = ["manzana", "platano", "naranja"]
print(f"'platano' in frutas: {'platano' in frutas}") # True
print(f"'uva' in frutas: {'uva' in frutas}") # False

# Combinación 'not in' / 'is not'
print(f"'uva' not in frutas: {'uva' not in frutas}") # True
print(f"a is not b: {a is not b}") # True
```

Terminal:



Name: Alan Souza
2DAM - SGE 28/10/26

# Act 5.0 Consola y Sobrecarga

```
import sys

#Ejemplo de lectura de argumentos desde la consola
def leer_argumentos_consola():
print("Nombre del script:", sys.argv[0])
print("Número de argumentos recibidos:", len(sys.argv) - 1)
print("Lista de argumentos:", sys.argv[1:])


#Ejemplo de "Sobrecarga de funciones"

#Sobrecarga con parámetros por defecto
def calcular_area(base, altura=None):
if altura is None:
# Si recibe 1 argumento, asume que es un cuadrado
return base * base
# Si recibe 2 argumentos, calcula el área de un rectángulo
return base * altura


#Sobrecarga con número indefinido de parámetros (*args)
def sumar_varios(*numeros):
"""Suma un número indefinido de parámetros pasados a la función."""
return sum(numeros)


if __name__ == "__main__":
print("--- Argumentos de Consola ---")
leer_argumentos_consola()

print("\n--- Sobrecarga con parámetros opcionales ---")
print("Área de cuadrado (lado 4):", calcular_area(4))
print("Área de rectángulo (base 4, altura 5):", calcular_area(4, 5))

print("\n--- Sobrecarga indefinida (*args) ---")
print("Suma de 2 números (5, 10):", sumar_varios(5, 10))
print("Suma de 5 números (1, 2, 3, 4, 5):", sumar_varios(1, 2, 3, 4, 5))
print("Suma sin parámetros:", sumar_varios())
```

Name: Alan Souza
2DAM - SGE 28/10/26

# Act 6.0 Ordenar y Key

```python
#Lista donde cada elemento es [medida/altura, peso]
personas = [
[180, 75],
[165, 60],
[180, 70], # Empate en altura con el primero, pero menor peso
[175, 80],
[190, 85],
[165, 55], # Empate en altura con el segundo, pero menor peso
]

"""
EXPLICACIÓN DE LA KEY FUNCTION:
La 'key function' recibe un único argumento (en este caso el elemento 'x' -> [altura, peso])
y devuelve una tupla que Python utiliza para establecer el orden de comparación.
Al devolver (-x[0], x[1]):
- Negamos la altura (-x[0]) para lograr un orden DESCENDENTE (cuanto mayor sea la altura,
menor será el valor negativo).
- Dejamos el peso positivo (x[1]) para lograr un orden ASCENDENTE en caso de empate.
"""
personas_ordenadas = sorted(personas, key=lambda x: (-x[0], x[1]))

print("Lista original:")
for p in personas:
print(f"Altura: {p[0]} cm, Peso: {p[1]} kg")
print("\nLista ordenada (Mayor altura, y a igualdad, menor peso):")
for p in personas_ordenadas:
print(f"Altura: {p[0]} cm, Peso: {p[1]} kg")
```

Terminal:



Name: Alan Souza
2DAM - SGE 28/10/26

# Act 7.0  Programación orientada a objetos

```
import random


class Car:

def __init__(self, matricula: int, color: str):
self.matricula = matricula
self.color = color
self.encendido = False

def imprimir(self):
print(
f"Coche -> Matrícula: {self.matricula:04d} | Color: {self.color}"
)

#Arrancar
def arrancar(self):
self.encendido = True
print(f"El coche con matrícula {self.matricula} ha arrancado.")

#Repintar
def repintar(self, nuevo_color: str):
self.color = nuevo_color
print(f"El coche {self.matricula} se ha pintado de {self.color}.")


def main():
colores_disponibles = ["red", "white", "black", "pink", "blue"]
coches = []

try:
n = int(input("Introduce el número de coches a crear (n): "))
except ValueError:
print("Debes introducir un número entero válido.")
return

#Creación de n instancias de la clase Car
for i in range(1, n + 1):
color_aleatorio = random.choice(colores_disponibles)
coche = Car(matricula=i, color=color_aleatorio)
coches.append(coche)
```

Name: Alan Souza
2DAM - SGE 28/10/26

```
#Limitar la impresión a un máximo de 10 instancias
limite = min(n, 10)
print(f"\n--- Imprimiendo las primeras {limite} instancias ---")
for i in range(limite):
coches[i].imprimir()


if __name__ == "__main__":
main()
```

Terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

 /bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/ptUno_act7.py
alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$ /bin/python3 /home/alasoucos2/Documents/Alan2DAM/SGE/UD00_parte1/ptUno_act7.py
 Introduce el número de coches a crear (n): 4

 --- Imprimiendo las primeras 4 instancias ---
 Coche -> Matrícula: 0001 | Color: black
 Coche -> Matrícula: 0002 | Color: white
 Coche -> Matrícula: 0003 | Color: red
 Coche -> Matrícula: 0004 | Color: red
alasoucos2@orj1:~/Documents/Alan2DAM/SGE/UD00_parte1$
```